

CSCI 490: LLM Foundations and AI Safety

**Hunter College, City
University of New York**

Department of Computer Science

CSCI 490: LLM Foundations and AI Safety

Special Topics in Artificial Intelligence

Course Number	CSCI 490
Credits	3 credits
Semester	Fall 2026
Format	Online Synchronous
Meeting Day	Tuesday
Meeting Time	10:00 AM - 1:00 PM ET
Platform	Zoom (link distributed via Brightspace)
Instructor	Konstantinos Krampis
Email	kk104@hunter.cuny.edu
Office Hours	[Days, Times] via Zoom — link in Brightspace
Response Policy	Emails answered within 48 hours on business days

This syllabus is subject to change. Any changes will be announced via Brightspace and confirmed in class.

Course Description

CSCI 490 is a project-centered course that builds practical and theoretical competencies in AI alignment research engineering. This course will immerse students in the methods and open problems of technical AI safety, covering the internals of large language models, mechanistic interpretability, reinforcement learning, and evaluation methodology. Over twelve weeks, students progress from deep learning fundamentals through transformer mechanistic interpretability, reinforcement learning, large language model (LLM) evaluation, and alignment science.

Prerequisites

- Proficiency in Python (equivalent to CSCI 127 or higher)
 - Familiarity with linear algebra and basic calculus (MATH 155 / MATH 150 or equivalent)
-

Course Learning Outcomes

Upon successful completion of this course, students will be able to:

1. **Implement** neural network components — including CNNs, ResNets, and a full backpropagation framework — from scratch in PyTorch, and train generative models such as VAEs and GANs.
2. **Build and train** GPT-2 style transformer language models end-to-end, implementing tokenization, multi-head attention, and autoregressive generation.
3. **Apply** mechanistic interpretability tools — including TransformerLens hooks, induction head detection, activation patching, indirect object identification (IOI) circuit analysis, function vectors, and sparse autoencoders (SAEs) — to reverse-engineer the internal computations of language models.
4. **Train** reinforcement learning agents using DQN, Vanilla Policy Gradient, and Proximal Policy Optimization (PPO), and implement a complete RLHF pipeline to align a language model to a human preference signal.
5. **Design and execute** LLM safety evaluations, including threat modeling, synthetic dataset generation, and running evaluations using the Inspect framework.
6. **Analyze** alignment-relevant failure modes — including emergent misalignment, deceptive alignment, sandbagging,

power-seeking, and sycophancy — through hands-on experiments with language models.

7. **Communicate** technical AI safety research findings in written and oral formats appropriate for a research audience.

Course Format — Online Synchronous

All sessions meet live via Zoom once per week during the scheduled meeting time. Attendance is expected for the full session. Each weekly session (~2 hours 30 minutes) is structured as follows:

Block	Duration	Activity
Lecture Part I	45 min	Instructor-led presentation and live demos
Break	15 min	—
Lecture Part II	45 min	Instructor-led presentation and live demos
Coding Lab	60 min	Students work on exercises; instructor circulates

Sessions will be recorded and posted to Brightspace within 24 hours for asynchronous review. Recordings do not substitute for live attendance.

Grading

Component	Weight
Weekly Coding Assignments (10 assignments, lowest 2 dropped)	60%
Quizzes (3 quizzes — Weeks 3, 6, 10)	30%
Participation & Discussion	10%

Assessment Descriptions

Weekly Coding Assignments: Each week's exercises are submitted by 11:59 PM ET on the day before the next class session via Brightspace. Assignments are graded on correctness and code quality.

Quizzes: Three short assessments (30–45 minutes) administered live via Brightspace at the start of class in Weeks 3, 6, and 10. Each covers material from the preceding unit. Closed-book; no external resources.

Participation: Evaluated on engagement during live sessions, contributions to the class discussion board in Brightspace, and peer code review activities.

Course Policies

Attendance

Because this course meets only once per week, each session carries significant weight. Attendance at all sessions is strongly expected. More than **two unexcused absences** will result in a one-letter-grade deduction. If you anticipate a conflict, contact the instructor in advance. Documented emergencies (illness, family emergency) will be handled individually.

Late Work

Assignments submitted up to 24 hours late receive a maximum of 70%. Assignments will not be accepted more than 24 hours late except in cases of documented emergency.

Academic Integrity

Hunter College regards acts of academic dishonesty (e.g., plagiarism, cheating, copyright infringement) as serious offenses against the values of intellectual honesty. The college is committed to dealing with such acts firmly and fairly. The penalty for an act of academic dishonesty may be a failing grade for the particular assignment and/or the course. Copies of the CUNY Academic Integrity Policy and the Hunter College Honor Code are available in the Provost's office.

Regarding AI tools: you may use AI coding assistants (GitHub Copilot, ChatGPT, Claude) as a learning aid to understand concepts and debug code. However, all submitted work must reflect your own understanding. Copying AI-generated solutions wholesale and submitting them without comprehension constitutes academic dishonesty.

Online Conduct

Students are expected to maintain professional behavior in Zoom sessions and Brightspace. This includes using your real name, keeping your camera on when feasible, and engaging respectfully. Hunter College has a zero-tolerance policy for bullying and cyberbullying.

College-Wide Statements

Accessibility

In compliance with the American Disability Act of 1990 (ADA) and with Section 504 of the Rehabilitation Act of 1973, Hunter College is committed to ensuring educational access and accommodations for all its registered students. Hunter College's students with disabilities and medical conditions are encouraged to register with the Office of AccessABILITY for assistance and accommodation. For information and appointment contact the Office of AccessABILITY located in Room E1214 or call (212) 772-4857 / TTY (212) 650-3230.

Title IX

Hunter College reaffirms its commitment to maintain an educational environment without sexual misconduct. Title IX of the Education Amendments of 1972 prohibits sex discrimination against any participant in an educational program or activity that receives federal funds. The act is intended to eliminate sex discrimination in education. Title IX covers discrimination based on sex or gender including sexual harassment, sexual assault, and sexual violence. If a student has a concern about such matters, they may call (212) 396-6080 or consult the [Hunter College Title IX page](#).

CUNY Policy on Sexual Misconduct

In compliance with the CUNY Policy on Sexual Misconduct, Hunter College reaffirms the prohibition of any sexual misconduct, which includes sexual violence, sexual harassment, and gender-based harassment retaliation against students, employees, and visitors, as well as certain intimate partner violence.

Weekly Course Schedule

Key: One synchronous session per week (~2 hours 30 minutes). Total: 12 sessions across the semester. Assignments are due by 11:59 PM ET the day before the following class session.

Unit I — Fundamentals of Deep Learning (Weeks 1-3)

Week 1 — PyTorch Fundamentals & Practical Deep Learning Skills

Topics: Core concepts review (neural networks, linear algebra, probability, information theory, KL divergence); einops/einsum tensor manipulation; batched tensor operations via a ray tracing exercise (parametrizing rays, batched intersection tests, 3D rendering); GPU-style parallelism; coding best practices and type annotations.

Assignment 1 Due Before Week 2 session

Outcomes: Students are fluent in PyTorch tensor operations, einops/einsum notation, and batched matrix computations.

Week 2 — CNNs, ResNets & Optimization

Topics: Building `nn.Module` subclasses; training loops with dataloaders and validation; implementing convolutions from scratch; batch normalization; assembling ResNet34 and loading pretrained ImageNet weights; feature extraction vs. fine-tuning; SGD, RMSprop, and Adam implemented from scratch; loss landscape visualization; Weights & Biases for experiment tracking and hyperparameter sweeps; introduction to distributed training with `torch.distributed` (data, pipeline, and tensor parallelism).

Assignment 2 Due Before Week 3 session

Outcomes: Students can build and train a ResNet from scratch, track and sweep experiments with W&B, and implement core optimizers by hand.

Week 3 — Backpropagation & Generative Models | Quiz 1

Topics: Computational graphs and automatic differentiation; implementing a complete autograd framework from scratch (forward pass, topological sort, backward pass); training an MLP on MNIST using custom backprop; variational autoencoders (VAEs) — encoder/decoder, ELBO loss, reparameterization trick; generative adversarial networks (GANs) — adversarial training dynamics, discriminator and generator objectives; transposed convolutions.

Quiz 1

Administered at start of class (Unit I — Ch0 material)

Assignment 3 Due

Before Week 4 session

Outcomes: Students can implement a working autograd framework and understand the training dynamics of both VAEs and GANs.

Unit II — Transformer Architecture & Mechanistic Interpretability (Weeks 4-8)

Week 4 — Transformer from Scratch

Topics: GPT-2 style decoder transformer: multi-head self-attention, positional encodings, layer normalization, MLP layers, residual stream; implementing the full forward pass in PyTorch following the TransformerLens architecture; tokenization; cross-entropy language modeling loss; autoregressive text generation with greedy, top-k, and nucleus (top-p) sampling.

Assignment 4 Due Before Week 5 session

Outcomes: Students can implement and train a GPT-style transformer from scratch and generate text from it.

Week 5 — Mechanistic Interpretability I: TransformerLens & Induction Heads

Topics: TransformerLens library: loading HookedTransformer models, caching activations, tokenizer usage; circuitsvis for attention pattern visualization; induction heads — identifying in-context learning circuits from attention patterns in a repeating sequence; hook-based ablation; factored QK and OV matrix analysis; reverse-engineering induction circuits from weights using composition scores.

Assignment 5 Due Before Week 6 session

Outcomes: Students can use TransformerLens to inspect model internals, identify induction heads programmatically, and reason about circuit behavior from both activations and weights.

Week 6 — Mechanistic Interpretability II: Probing, Function Vectors & Model Steering | Quiz 2

Topics: Linear probes on residual stream activations to detect internal representations; probing for deceptive reasoning; nnsight library for causal interventions on large models (GPT-J-6B); task-encoding hidden states (h-vectors); reproducing function vector results — that residual stream contains task-encoding vectors that induce zero-shot behavior; steering vectors in GPT2-XL — replicating behavioral changes from Turner et al.

Quiz 2	Administered at start of class (Unit II Weeks 4–5 material)
Assignment 6 Due	Before Week 7 session

Outcomes: Students can train probes on language model activations, perform causal interventions with nnsight, and steer model behavior via activation addition.

Week 7 — Mechanistic Interpretability III: Sparse Autoencoders & IOI Circuits

Topics: Sparse autoencoders (SAEs): theory, training objective (L1 + reconstruction loss), monosemantic feature extraction, feature visualization; interpretability with SAEs on real models; indirect object identification (IOI) — full circuit replication in GPT-2 Small including name mover heads, duplicate token heads, and induction heads; path patching; logit lens and logit attribution.

Assignment 7 Due Before Week 8 session

Outcomes: Students can train and interpret a sparse autoencoder and replicate a published circuit-level analysis in a real language model.

Week 8 — Mechanistic Interpretability IV: Superposition & Toy Models

Topics: The superposition hypothesis: why neural networks represent more features than dimensions; polysemanticity; feature geometry and the role of sparsity; training SAEs on toy superposition models; implications of superposition for scalable interpretability; connection to monosemanticity research at Anthropic.

Assignment 8 Due Before Week 9 session

Outcomes: Students can explain the superposition hypothesis, train toy SAEs, and connect feature geometry to the challenges of interpretability at scale.

Unit III — Reinforcement Learning & RLHF (Weeks 9-10)

Week 9 — RL Fundamentals, DQN & Policy Gradients

Topics: Markov decision processes; policies, value functions, and Bellman equations; analytic solutions (policy iteration, value iteration); sampling-based methods (multi-armed bandits, ϵ -greedy, UCB); Deep Q-Networks (DQN) with experience replay and target networks; OpenAI Gym environments; Vanilla Policy Gradient (VPG) with baseline and reward-to-go.

Assignment 9 Due Before Week 10 session

Outcomes: Students can implement and train DQN and VPG agents on standard Gym environments and articulate the exploration-exploitation tradeoff.

Week 10 — PPO & Reinforcement Learning from Human Feedback | Quiz 3

Topics: Proximal Policy Optimization (PPO): clipped surrogate objective, entropy bonus, generalized advantage estimation (GAE); implementing and training a PPO agent; reward model training from human preference comparisons; complete RLHF pipeline — supervised fine-tuning, reward modeling, and PPO-based alignment applied to a transformer language model.

Quiz 3

Administered at start of class
(Unit III Week 9 material)

Outcomes: Students can train a PPO agent from scratch and implement an end-to-end RLHF loop to align a language model to a human preference signal.

Unit IV — LLM Evaluation & Alignment Science (Weeks 11-12)

Week 11 — LLM Evaluation: Threat Modeling, Dataset Generation & Inspect

Topics: Introduction to LLM safety evaluations and why they matter; LLM API usage basics; alignment faking as a worked case study; threat modeling and safety cases; evaluation specification and design; synthetic dataset construction and quality control; running evaluations at scale using the UK AISI Inspect framework; interpreting and communicating eval results.

Outcomes: Students can design a threat model, generate a synthetic evaluation dataset, and execute a model evaluation using the Inspect framework.

Week 12 — LLM Agents & Alignment Science

Topics: LLM agent architectures: tool use, agent loops, multi-step reasoning; agent evaluation methodologies; emergent misalignment in language models — power-seeking, sycophancy, deceptive alignment, sandbagging, corrigibility, and self-preservation; theoretical frameworks for the science of misalignment; mechanistic analysis of chain-of-thought and reasoning model internals; LLM psychology and persona vectors.

Outcomes: Students can build and evaluate a basic LM agent and analyze alignment failure modes with reference to the experimental and theoretical literature.

Key Dates

Week	Event
Week 1	

Week	Event
	First class session; environment setup verified in class
Week 3	Quiz 1 (Unit I — Fundamentals)
Week 6	Quiz 2 (Unit II Weeks 4-5 — Transformer Architecture & Mech Interp I)
Week 10	Quiz 3 (Unit III Week 9 — RL Fundamentals)

Recommended Background Reading

These are not required but will deepen your understanding:

- Elhage et al. (2021). A Mathematical Framework for Transformer Circuits. Anthropic.
- Wang et al. (2022). Interpretability in the Wild: a Circuit for Indirect Object Identification in GPT-2 small. ICLR.
- Christiano et al. (2017). Deep Reinforcement Learning from Human Preferences. NeurIPS.
- Hubinger et al. (2019). Risks from Learned Optimization in Advanced Machine Learning Systems. MIRI.
- Templeton et al. (2024). Scaling Monosemanticity. Anthropic.
- Turner et al. (2023). Activation Addition: Steering Language Models Without Optimization. arXiv.

Hunter College is committed to providing equal access to its programs for all students. For more information on college policies, resources, and student services, visit hunter.cuny.edu.